

Classification Preprocessing

Encoding and scaling patterns for classification datasets

The raw dataset

Data overview

DESCRIPTION

A typical classification dataset mixes two column types: float64 columns are continuous numeric measurements (e.g. age, height, weight), and object columns are categorical strings (e.g. yes/no flags, transport mode, frequency labels). ML models require all inputs to be numbers on a comparable scale, so each type needs its own treatment: scale the floats, encode the strings, and label-encode the target column separately.

SAMPLE DATA

Gender	Age	Height	Weight	family_history_with_overweight	FAVC	FCVC	NCP	CAEC	SMOKE	CH20	SCC
Female	21.0	1.62	64.0	yes	no	2.0	3.0	Sometimes	no	2.0	no
Female	21.0	1.52	56.0	yes	no	3.0	3.0	Sometimes	yes	3.0	yes
Male	23.0	1.80	77.0	yes	no	2.0	3.0	Sometimes	no	2.0	no
Male	27.0	1.80	87.0	no	no	3.0	3.0	Sometimes	no	2.0	no
Male	22.0	1.78	89.8	no	no	2.0	1.0	Sometimes	no	2.0	no

CODE

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, OneHotEncoder

data = pd.read_csv("dataset.csv")
print(data.dtypes)  # float64 columns = continuous, object columns = categorical
```

Scale continuous features

Numerical scaling

DESCRIPTION

Continuous columns often live on very different scales — one feature might range from 0–1 while another ranges from 0–200. Many ML algorithms use distance or gradient calculations that treat large numbers as more important simply because they're bigger, not because they carry more signal. `StandardScaler` removes that bias: it shifts each column to $\text{mean}=0$ and scales it so one standard deviation equals 1, making every feature contribute equally.

CODE

```
# select_dtypes(float64) picks our continuous columns without a manual list
continuous_columns = data.select_dtypes(include=['float64']).columns.tolist()

# StandardScaler: for each column subtract the mean, divide by std deviation
# After this step every column has mean≈0 and std≈1
scaler = StandardScaler()

# fit_transform: learns mean/std FROM this data, then applies the scaling
# On a test set call scaler.transform() only — never fit on test data
scaled_features = scaler.fit_transform(data[continuous_columns])

# numpy arrays lose column names, so we wrap the result back in a DataFrame
scaled_df = pd.DataFrame(
    scaled_features,
    columns=scaler.get_feature_names_out(continuous_columns)
)

# Drop the original float columns and reattach the scaled versions
# axis=1 means "join side by side" (columns), not stacked (rows)
scaled_data = pd.concat(
    [data.drop(columns=continuous_columns), scaled_df],
    axis=1
)
```

One-hot encode categorical inputs

Categorical encoding

DESCRIPTION

Models can't do maths on strings like 'yes'/'no' or 'Public_Transportation'. One-hot encoding turns each category into its own 0/1 column. `drop='first'` removes one column per feature to avoid the dummy variable trap — if you keep all dummies, one is always a perfect linear combination of the others, which breaks linear models. The target column is excluded here because it is what we're predicting, not an input.

CODE

```
# Re-detect string columns on scaled_data (only categorical ones remain)
categorical_columns = scaled_data.select_dtypes(include=['object']).columns.tolist()

# Remove the target column – encoding it as an input would be a data-leakage mistake
TARGET = 'NObeyesdad'
categorical_columns.remove(TARGET)

# drop='first' avoids the dummy variable trap:
# with k categories we only need k-1 columns; the dropped one is implied by the others
# e.g. Gender keeps only 'Male' (1=male, 0=female); no information is lost
encoder = OneHotEncoder(sparse_output=False, drop='first')
encoded_features = encoder.fit_transform(scaled_data[categorical_columns])

# Restore column names so the DataFrame stays readable downstream
encoded_df = pd.DataFrame(
    encoded_features,
    columns=encoder.get_feature_names_out(categorical_columns)
)

# Replace the original string columns with their numeric encoded counterparts
prepped_data = pd.concat(
    [scaled_data.drop(columns=categorical_columns), encoded_df],
    axis=1
)
```

Label-encode the target column

Categorical encoding

DESCRIPTION

The target column contains class labels as strings. We use label encoding — assigning each class a single integer — rather than one-hot encoding, because classifiers expect one output column of class indices, not multiple binary columns. pandas `.cat.codes` is deterministic: it always assigns integers in alphabetical order, so the mapping is stable and reproducible across runs.

CODE

```
# astype('category') registers the unique class strings as an ordered set
# .cat.codes maps each to a stable integer: 0, 1, 2, ... (alphabetical order)
prepped_data[TARGET] = (
    prepped_data[TARGET]
    .astype('category')
    .cat.codes
)

# To decode predictions back to class names later:
# label_map = dict(enumerate(df[TARGET].astype('category').cat.categories))
# label_map[0] → original class string
```

Full pipeline — preprocess_data(df)

Feature engineering

DESCRIPTION

All three steps wrapped into a single reusable function. Wrapping preprocessing in a function guarantees the steps always run in the correct order and nothing is accidentally skipped. The function is self-contained — it creates and fits its own scaler and encoder. For production use you would separate fitting (on train) from transforming (on test), but for a single dataset this keeps things clean and simple.

CODE

```
def preprocess_data(df, target_column):
    """
    Standard preprocessing pipeline for a classification dataset.

    Steps applied (in order):
    1. StandardScaler on all float64 columns (zero mean, unit variance)
    2. OneHotEncoder (drop='first') on all object columns except the target
    3. Label encoding (integer codes) on the target column

    Parameters
    -----
    df : pd.DataFrame - raw dataset
    target_column : str - name of the column to predict

    Returns
    -----
    pd.DataFrame - fully numeric, ready for model training
    """
    # — 1. Scale continuous features —————
    continuous_columns = df.select_dtypes(include=['float64']).columns.tolist()
    scaler = StandardScaler()
    scaled_df = pd.DataFrame(
        scaler.fit_transform(df[continuous_columns]),
        columns=scaler.get_feature_names_out(continuous_columns)
    )
    scaled_data = pd.concat([df.drop(columns=continuous_columns), scaled_df], axis=1)

    # — 2. One-hot encode categorical inputs —————
    categorical_columns = scaled_data.select_dtypes(include=['object']).columns.tolist()
    categorical_columns.remove(target_column)
    encoder = OneHotEncoder(sparse_output=False, drop='first')
    encoded_df = pd.DataFrame(
        encoder.fit_transform(scaled_data[categorical_columns]),
        columns=encoder.get_feature_names_out(categorical_columns)
    )
    prepped_data = pd.concat(
        [scaled_data.drop(columns=categorical_columns), encoded_df],
        axis=1
    )

    # — 3. Label-encode the target —————
    prepped_data[target_column] = (
        prepped_data[target_column].astype('category').cat.codes
    )

    return prepped_data
```